MiniGPT-4(2023, KAUST)

> MiniGPT-4：LLM 辅助多模态 (2023.04, KAUST)
> MiniGPT-4: Enhancing Vision-Language Understanding with Advanced Large Language Models

MiniGPT-4 是对 GPT-4 的多模态能力的一次探索和尝试，其实也是和 LLaVA 相同的目的，都认为增强多模态模型的生成能力一定要利用 LLM 的知识库。因此也是采用“对齐”的思路来训练一个投影层将预训练好的 LLM 和视觉模型链接在一起。

核心点：

- 固定 vision encoder 和 LLM，仅训练 projection 对齐部分，就能获得不错的效果，并且仅用一个 projection layer 就能实现很强大的对齐功能
- 仅使用 image caption 进行对齐并不能获得很好的效果，而用小部分的 detailed image description pairs 进行 further finetuning 就能打破这个限制
- MiniGPT-4 表现出了和 GPT-4 类似的能力，比如生成复杂的 image descriptions，从手写文本生成网页，还有 GPT-4 没有的能力，比如从食物照片生成详细的烹饪食谱，根据图像编写故事或诗歌，为图像中的产品编写广告等

1. 模型结构

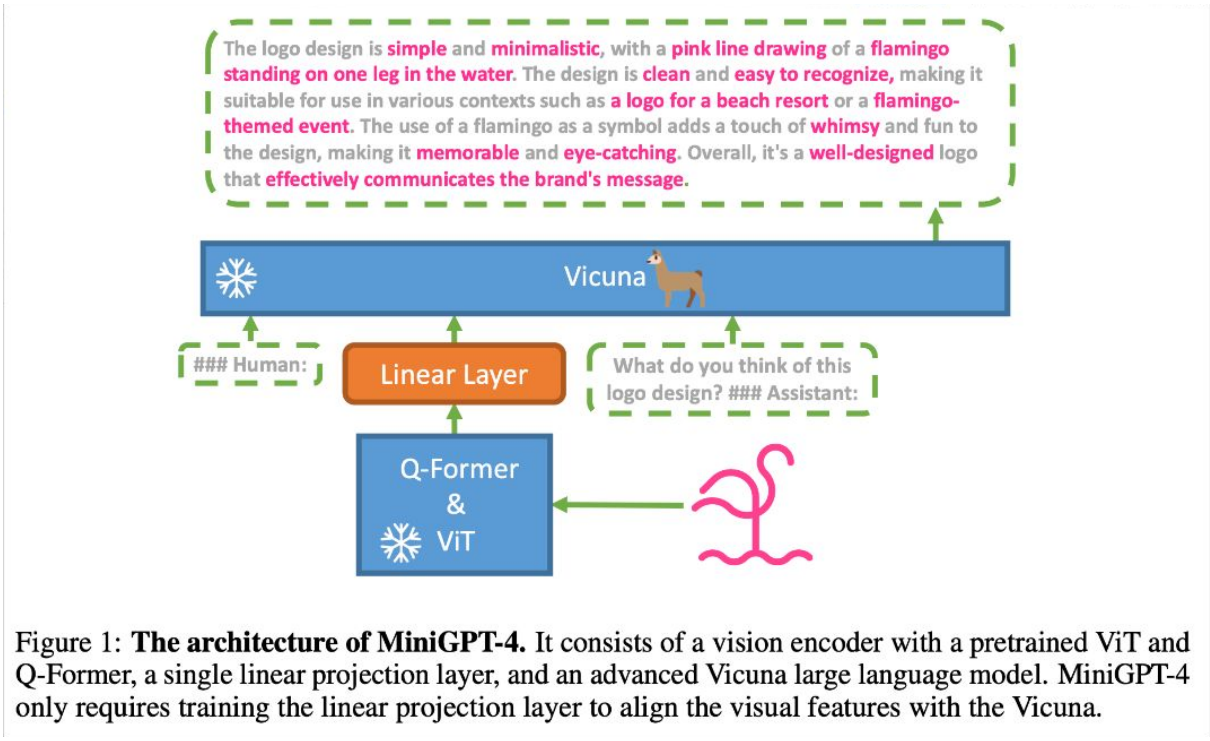


Figure 1: **The architecture of MiniGPT-4.** It consists of a vision encoder with a pretrained ViT and Q-Former, a single linear projection layer, and an advanced Vicuna large language model. MiniGPT-4 only requires training the linear projection layer to align the visual features with the Vicuna.

- **视觉模块 Image Encoder**：
  预训练好的BLIP-2中的视觉部分 ViT + Q-former（⚠ Q-former 这里只有视觉的输入，没有文本的输入）
- **对齐模块 Projection layer**：
  一个线性层 linear。
  该线性层实际上是一个升维操作，将 ViT 输出的编码向量 768 维变为 4096 维（在 BLIP-2 中这里是 2560）
- **语言模块 LLM**：
  预训练好的 Vicuna（和 LLaVA 用的一样）。

image encoder 和 LLM 都不参与训练，仅训练 projection 的参数，降低了计算开销，训练也很高效。从上述架构可见：miniGPT4 的输入是：图片和文本，输出是文本序列。

创建 projection 的代码：

```
# 创建线性投影层，将经过 Q-Former 转换的图像特征映射到语言模型的嵌入空间
# img_f_dim 是图像特征的维度
# llama_model.config.hidden_size 是语言模型的隐藏状态的维度
self.llama_proj = nn.Linear(img_f_dim, self.llama_model.config.hidden_size)
```

输入图像后 MiniGPT-4 完整的处理流程：

```
def encode_img(self, image):device = image.devicewith self.maybe_autocast():
    # 使用视觉编码器对图像进行编码，再使用 LayerNorm 标准化
    image_embeds = self.ln_vision(self.visual_encoder(image)).to(device)
    # 默认使用冻结的 Q-Former
    if self.has_qformer:
    # 创建图像的注意力掩码
        image_atts = torch.ones(image_embeds.size()[:-1], dtype=torch.long).to(device)
        # 扩展查询标记以匹配图像特征的维度
        query_tokens = self.query_tokens.expand(image_embeds.shape[0], -1, -1)
        # 使用 Q-Former 模块计算查询标记和图像特征的交叉注意力，更好地对齐图像和文本
        query_output = self.Qformer.bert(query_embeds=query_tokens,encoder_hidden_states=
    image_embeds,encoder_attention_mask=image_atts,return_dict=True,)
        # 通过线性投影层将 Q-Former 的输出映射到语言模型的输入
        inputs_llama = self.llama_proj(query_output.last_hidden_state)
        # 创建语言模型的注意力掩码
        atts_llama = torch.ones(inputs_llama.size()[:-1], dtype=torch.long).to(image.device)
    # 返回最终输入进语言模型中的图像编码和注意力掩码
    return inputs_llama, atts_llama
```

输入流程概括：

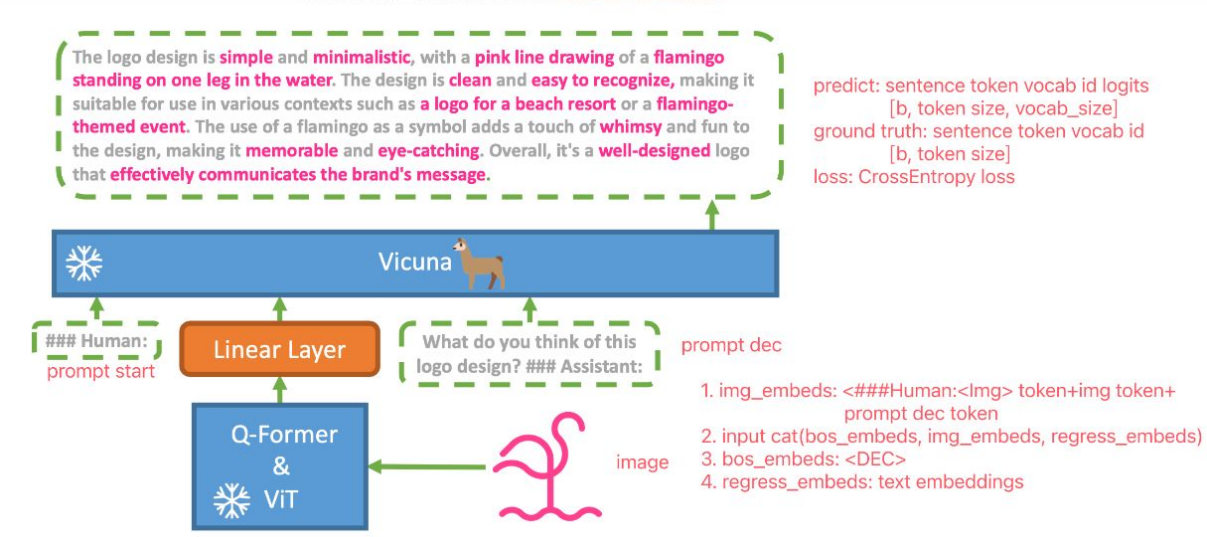
输入的图片经过视觉模块进行编码得到一个图像 embedding，然后将图像 embedding 经过一个线性层与文本 embedding 对齐，再将原文本 embedding 和经过对齐后的图像 embedding concat 起来，输入到 LLM 中，就可以实现能够接受多模态信息的 GPT 了。

> 在 LLaVA 中之用 projection layer 方法也尝试过了，也是比较有用的，但 LLaVA 还更新了 LM 的参数，但是 MiniGPT-4 没有，那性能上会有差距吗？
> 我感觉 MiniGPT-4 在图像多轮对话上的能力应该是比 LLaVA 要差的，因为 LLaVA 用图片多轮对话训练过 LM，肯定是有性能提升的。但是图像理解能力可能差距不大，或者 MiniGPT-4 还更好点。

2. 训练方法

MiniGPT-4 也是**两阶段**的训练方法：**模型预训练和模型微调**

- Stage-1 预训练，MiniGPT-4 在大量的图像-文本对数据上进行预训练，以获得基础的视觉语言知识；
- Stage-2 微调，MiniGPT-4 使用数量更少但质量更高的图像-文本数据集进行微调，以进一步提高预训练模型的生成质量与综合表现。



2.1 Stage-1 预训练

- 目标：模型需要从大量对齐的图像-文本对中获取视觉-语言知识，loss 是交叉熵。
- LLM 的输入（3 部分组成）：
  句子开始标志符编码 bos_emds + 图像编码特征 img_embeds + 图像对应的描述句子编码 regress_embeds
  没有上图中 2 个绿色部分输入（prompt start 和 prompt dec），只有 Linear Layer 的输出作为输入，加上文本 tokenizer 进行 embedding 之后，连接在一起输入给 LLM
- Projection 的输出可以看作是 LLM 的 soft prompt，提示 LLM 生成相应的真实文本。
- 在整个预训练过程中，预训练的**视觉编码器和 LLM 都保持冻结状态**，**只有线性投影层被用来预训练**。
- 使用的数据集包括 Conceptual Caption、SBU 和 LAION。

训练了 2w 步左右，批量大小为 256，覆盖大约 500 万个图像文本对。整个过程大约需要 10 个小时才能完成，使用 4 张 A100 (80GB) GPU。

但是，在第一阶段预训练完成后，MiniGPT-4 有一定的图文响应能力，但是生成并不流畅，会有**重复说话，句子碎片化，输出不相干**等问题 (这也是多模态大模型的常见问题了)。

输入 embedding 代码：

```
# img_embeds: vit+Q-former+line proj输出
img_embeds, atts_img = self.encode_img(image)

# 对句子的开始标志编码
# embed_tokens = nn.Embedding(config.vocab_size, config.hidden_size, self.padding_idx),用于对词向
    量降维
bos_embeds = self.llama_model.model.embed_tokens(bos)


# text为图像的文本描述
text = [t + self.end_sym for t in samples["text_input"]]
to_regress_tokens = self.llama_tokenizer(
            text,
            return_tensors="pt",
            padding="longest",
            truncation=True,
            max_length=self.max_txt_len, #默认160,只编码句子的一部分
            add_special_tokens=False
```

```
        ).to(image.device)  # text to tokens

# 对文本tokenizer之后的词id进行编码，也传入作为embed去训练
to_regress_embeds = self.llama_model.model.embed_tokens(to_regress_tokens.input_ids)
inputs_embeds = torch.cat([bos_embeds, img_embeds, to_regress_embeds], dim=1)
```

下面一段示例代码，有助于更好地理解 MiniGPT-4 的训练过程，实现了整个 MiniGPT-4 模型的前向传播过程，包括图像和文本的编码、提示处理、多模态数据编码的连接，以及最终损失的计算。

```
def forward(self, samples):
    image = samples["image"]
    # 对输入图像进行编码
    img_embeds, atts_img = self.encode_img(image)
    # 生成文本指令
    instruction = samples["instruction_input"] if "instruction_input" in samples else None
    # 将指令包装到提示中
    img_embeds, atts_img = self.prompt_wrap(img_embeds, atts_img, instruction)
    # 配置 tokenizer 以正确处理文本输入
    self.llama_tokenizer.padding_side = "right"text = [t + self.end_sym for t in samples["answer"
    ]]
    # 使用 tokenizer 对文本进行编码
    to_regress_tokens = self.llama_tokenizer(text,return_tensors="pt",padding="longest",
    truncation=True,max_length=self.max_txt_len,add_special_tokens=False).to(image.device)
    # 获取 batch_size
    batch_size = img_embeds.shape[0]
    # 创建开始符号的嵌入向量和注意力掩码
    bos = torch.ones([batch_size, 1],dtype=to_regress_tokens.input_ids.dtype,device=
    to_regress_tokens.input_ids.device) * self.llama_tokenizer.bos_token_idbos_embeds = self.
    embed_tokens(bos)atts_bos = atts_img[:, :1]
    # 连接图像编码、图像注意力、文本编码和文本注意力
    to_regress_embeds = self.embed_tokens(to_regress_tokens.input_ids)
    inputs_embeds, attention_mask, input_lens = self.concat_emb_input_output(img_embeds, atts_img
    , to_regress_embeds, to_regress_tokens.attention_mask)
    # 获得整体的输入编码和注意力掩码
    inputs_embeds = torch.cat([bos_embeds, inputs_embeds], dim=1)
    attention_mask = torch.cat([atts_bos, attention_mask], dim=1)
    # 创建部分目标序列，替换 PAD 标记为-100
    part_targets = to_regress_tokens.input_ids.masked_fill(to_regress_tokens.input_ids == self.
    llama_tokenizer.pad_token_id, -100)
    # 创建完整的目标序列，用于计算损失
    targets = (torch.ones([inputs_embeds.shape[0], inputs_embeds.shape[1]], dtype=torch.long).to(
    image.device).fill_(-100))
    for i, target in enumerate(part_targets):
        targets[i, input_lens[i] + 1:input_lens[i] + len(target) + 1] = target
    # 在自动混合精度环境下，计算语言模型的输出
    with self.maybe_autocast():
        outputs = self.llama_model(inputs_embeds=inputs_embeds,attention_mask=attention_mask,
    return_dict=True,labels=targets,)loss = outputs.loss
    # 返回损失作为输出
    return {"loss": loss}
```

2.2 构建微调数据集

为了优化效果，需要做第二阶段的对齐。为此 MiniGPT-4 专门构建了一个质量更高的指令数据集。

1. 作者从 Conceptual Caption 数据集随机挑选 5000 图片。用 stage-1 得到的模型基于下面的 prompt 生成详

细的图像描述。

<ImageFeature> 是上面提到的 soft visual token

```
###Human: <Img><ImageFeature></Img>Describe this image in detail. Give as many details as
    possible. Say everything you see. ###Assistant:
```

如果生成的结果不到 80 个 token，会增加额外的 prompt ###Human: Continue ###Assistant:，以确保图片描述尽可能的详细。

2. 但是上面生成的结果里包含了噪声或者不连贯或者语法错误的内容，因此使用 ChatGPT 来修复这些问题。

   给 ChatGPT 的 prompt 如下：

```
Fix the error in the given paragraph.
Remove any repeating sentences, meaningless characters, not English sentences, and so on.
Remove unnecessary repetition.
Rewrite any incomplete sentences.
Return directly the results without explanation.
Return directly the input paragraph if it is already correct without explanation.
```

3. 获得结果之后，再利用一些人工制定的规则进行筛选，比如包含“I′ m sorry I made a mistake...”，或者“I apologize for that ...”的回复可以删掉。
4. 最终得到了 3500 条高质量的数据，用于第二阶段的训练。

2.3 Stage-2 微调

在 stage-2 微调中，模型在上面生成的高质量图像-文本数据集上进行微调，仍然只训练 Linear Layer，也是优化 Generation Loss。

⚠ prompt 的损失不算入训练中

第二阶段由于数据非常少，使用单个 A100 只要 7 分钟就训练完了。

2.4 Ablation

作者设计了三个实验，研究除了 fine-tune 新加的 Linear layer，fine-tune 其他的模块有用吗？

1. 移除 Q-former，直接用 Projector map ViT 的特征到 Vicuna
2. 用三层 MLP，而不是一层
3. Fine-tune Projector 的同时也 fine-tune Q-former

结果如下：

Table 4: Ablation on architecture designs

Model	AOK-VQA	GQA
MiniGPT-4	58.2	32.2
(a) MiniGPT-4 w/o Q-Former	56.9	33.4
(b) MiniGPT-4 + 3 Layers	49.7	31.0
(c) MiniGPT-4 + Finetune Q-Former	52.1	28.0

Table 5: Hallucination Evaluation

	$CHAIR_i$	Avg. Length
Blip-2	1.3	6.5
MiniGPT-4 (short)	7.2	28.8
MiniGPT-4 (long)	9.6	175

这个结果说明：

1. 复杂的 Q-former 结构对 alignment 帮助不一定大，一层 MLP Projector 足矣；
2. 深层 MLP 映射不一定有用；
3. 原本 Align 过的 Q-former 如果在小的数据集上 fine-tune，效果甚至可能适得其反。

3. 总结

其实 MiniGPT-4 没有特别大的创新，但是做了非常多实验，总结了非常多的经验，他的 Ablation 其实说明了很大的两点内容：

1. 高质量数据非常关键，质量 > 数量
2. 模态对齐模块不一定要复杂，数据才是关键